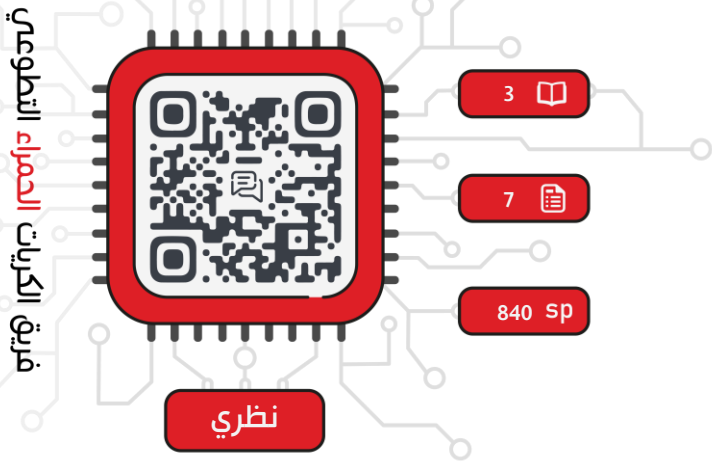


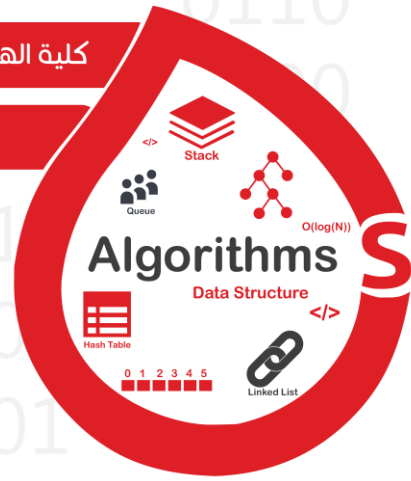


كلية الهندسة المعلوماتية

السنة الثانية

طرق التفكير

د. أبي صندوق



RB Informatics; 10/11/2024

خوارزميات 1

ذكرنا في المحاضرة السابقة أن الهدف من مادة الخوارزميات هو إيجاد طرق تفكير مختلفة لحل مسألة ما ، لذا سنتحدث اليوم عن طرق التفكير وأنواع الخوارزميات .

أولاً : Decrease and conquer

لها ثلاث أشكال :

1. Decrease by one

2. Decrease by factor

3. Decrease by random factor

وفيها نقوم بحل المشكلة الأساسية عن طريق تصغيرها إلى مشكلة فرعية وحل المشكلة الفرعية ثم تعميم الحل على المشكلة الأساسية

أي إذا كانت المشكلة من حجم (n) واستطعنا حلها لحجم $(n-1)$ هل يمكن تعميم هذا الحل لحجم (n) ؟



ملاحظة أرييسزية :

قد تُذكر أنواع الـ Decrease and conquer بأسماء أخرى:

- decrease by one $\Rightarrow$ decrease by constant
- decrease by factor $\Rightarrow$ decrease by a constant factor
- decrease by random factor $\Rightarrow$ variable size decrease

Decrease by one

مثال 1 :

نفرض لدينا عدد زوجي من المربعات نريد توزيعهم في كتلة واحدة (لا يوجد مربع منعزل لا يجاوره مربع آخر) ، وكل مربع يجاور عدد فردي من المربعات .

نجد أنه من الصعب إيجاد حل فوراً من أجل أي عدد زوجي من المربعات لذا نفكر بحل من أجل قيمة معينة ولتكن 20 مربع مثلاً

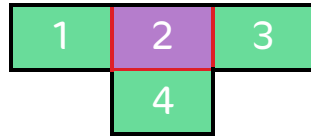
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

لتصغير المسألة سنفكر بحل من أجل 18 مربع ونعممه من أجل 20 مربع

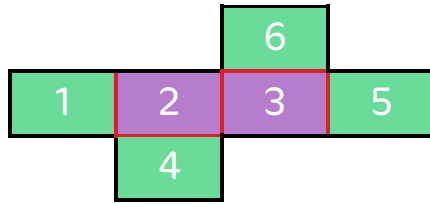
ولكن :

لإيجاد الحل من أجل 18 مربع لن يكون سهلاً ، لنحاول من أجل 16 أو 14 أو حتى من أجل 4 مثلاً

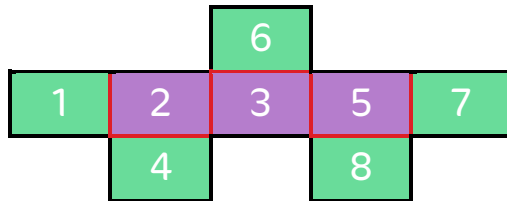
▪ من أجل 4 مربعات :



▪ من أجل 6 مربعات :



▪ من أجل 8 مربعات :



وهكذا

نلاحظ أنه :

بإيجادنا الحل من أجل 4 يمكننا تعميم الحل من أجل 20 أي نحاول إيجاد الحل بالجزء الصغير الذي يمكننا حله ثم تعميم

الحل على المشكلة الأساسية .

■ ملاحظة :

نقول by one لأننا نصغر المشكلة الأساسية إلى مشكلة فرعية أصغر بعدد ثابت من العناصر أي : إيجاد الحل من أجل $(n-c)$ وتعميمه من أجل n .

مثال 2 :

لتكن لدينا المصفوفة $A[10]$ ونريد إيجاد خوارزمية فرز (sort) بطريقة Decrease and conquer (يمكن حل المسألة بأسلوب عودي أو تكراري لكن سنستخدم التكراري).

نلاحظ أنه :

إذا كانت المسألة محلولة من أجل 9 عناصر سنكون قادرين على حلها من أجل 10 عناصر وذلك بإقحام العنصر غير المرتب بمكانه الصحيح .

ولكن :

حتى نحلها من أجل 9 أو 8 أو 7 أو عناصر سيكون صعباً ، لذلك سنفرض أن العنصر الأول من المصفوفة مرتب بمكانه الصحيح، ونحاول إقحام بقية العناصر بمكانها الصحيح :

الكود بلغة JAVA :

الشرح :

```
1. void sort (int []A){
2.   int k,key,i;
3.   for (k=1;k< A.length;k++){
4.     key = A[k];
5.     i=k;
6.     while(i>0 && A[i-1]>key){
7.       A[i]=A[i-1];
8.       i=i-1;
9.     }
10.    A[i]=key;
11.  }
12.}
```

السطر 3 : حلقة للمرور على جميع عناصر المصفوفة وترتيبهم ($k=1$ لأننا فرضنا أن $k=0$ مرتب).

السطر 4-5 : تخزين العنصر الحالي في key والـ index الخاص به في i

السطر 6 : طالما العنصر السابق أكبر من العنصر الحالي (والشرط $i>0$ حتى نضمن عدم الخروج خارج حدود المصفوفة)

السطر 7 : إذا تحقق الشرط (أي كان العنصر السابق < العنصر الحالي) ، يجب أن نحرك العنصر السابق إلى الأمام $A[i]=A[i-1]$.

السطر 8 : نحرك الـ index (i) للخلف لنستمر بمقارنة العنصر (key) مع جميع العناصر السابقة حتى نجد مكانه الصحيح .

السطر 10 : عند انتهاء حلقة الـ while نكون قد وصلنا للمكان الصحيح للعنصر لذا نقوم بإدراجه $A[i]=key$.

مثال توضيحي:

لدينا المصفوفة 5,2,1

↙ عند $k=2$

▪ الحلقة while :

2,5,5

2,2,5

▪ السطر العاشر :

1,2,5

↙ عند $k=1$

▪ الحلقة while :

5,5,1

▪ السطر العاشر :

2,5,1

وبالتالي تنتج لدينا المصفوفة المرتبة [1,2,5]

أي : نقارن كل عنصر بكل العناصر قبله حتى نجد مكانه الصحيح .التعقيد في أفضل حالة $O(n)$ بفرض العناصر مرتبة تصاعدياً .التعقيد في أسوأ حالة $O(n^2)$ بفرض العناصر مرتبة تنازلياً .

ملاحظة :

▪ تدعى الطريقة السابقة insertion sort (الفرز بالإقحام).

▪ avg التعقيد :

1. حلقة الـ for حتماً ستكرر n مرة2. حلقة الـ while بأفضل حالة ستكرر 0 مرة وأسوأ حالة n مرة أي بشكل متوسط $\frac{n}{2}$ مرة وبالتالي

$$n * \frac{n}{2} = n^2$$

تكون هذه الطريقة جيدة في حال كانت المصفوفة شبه مرتبة .

مثال واقعي عن عناصر شبه مرتبة :

↙ ترتيب الـ packets

packet : وحدة أو عنصر تستخدم كجزء من تصميم صفحة الويب

تم إرسال packets للمستقبل ولكن عند وصولهم لم يكونوا بالترتيب الصحيح ولكن بفضل التسلسل الزمني للإرسال وتأخر بعضهم عن بعض قليلاً وصلوا شبه مرتبين (ليسوا عشوائيين بشكل كلي) و لإعادة ترتيبهم وتكوين الصفحة نستخدم الطريقة السابقة .

Decrease by factor

مثال 3 :

بفرض نريد إيجاد x^a بأقل كلفة ممكنة .إذا كان a عدد زوجي :

$$x^a = \underbrace{x \times x \times x \dots \times x}_{a \text{ مرة}} \quad \text{فبدلاً من حساب}$$

$$x^a = (\underbrace{x \times x \times x \times \dots \times x}_{\frac{a}{2} \text{ مرة}})^2 \quad \text{يمكننا حساب}$$

$$x^a = (x^{\frac{a}{2}})^2 \quad \text{أي :}$$

هكذا قمنا بتقليل عمليات الجداء من a إلى $\frac{a}{2}$

$$x^a = ((x^{\frac{a}{4}})^2)^2 \quad \text{ولحساب } x^{\frac{a}{2}} \text{ نقوم بتكرار العملية السابقة :}$$

هكذا أصبحنا تقريباً بحاجة إلى $\log_2 a$ عملية للحصول على الناتج النهائي .إذا كان a عدد فردي :

$$x^a = x \cdot x^{a-1} \quad \text{أصبح زوجي الآن}$$

ونكرر الخطوات السابقة بنفس الطريقة .

■ ملاحظة:

نقول by factor لأننا في كل مرة نقلل عمليات الجداء من a إلى $\frac{a}{2}$ أي بالتقسيم.

ملاحظة أرييسزية:

إن الطريقة decrease by factor نسعى فيها إلى إيجاد طريقة بحيث نقوم بتصغير المسألة من n إلى $\frac{n}{\text{factor}}$ وقد يكون ال factor يساوي 2,3,4
نستطيع من خلالها جعل المسألة محلولة من أجل n وفي المثال السابق كان ال factor = 2 .

التعقيد بأفضل الحالات $\log_2 a$ (إذا كان $a=16$ مثلاً)التعقيد بأسوء الحالات $2\log_2 a$ (إذا كان $a=15$ مثلاً) ؛ لأنه في كل مرة نقسم a على 2 ونضرب بـ x

$$x^{15} = x \cdot x^{14} \Rightarrow x \cdot (x^7)^2$$

$$x^7 = x \cdot x^6 \Rightarrow x \cdot (x^3)^2 \dots$$

تهمل ال 2 في $2\log_2 a$ لاحقاً .

مثال 4 :

خوارزمية البحث Binary search

سنقوم بالبحث عن عنصر باستخدام هذه الخوارزمية

■ ما هو الـ Binary search ؟

هو خوارزمية فعالة للبحث عن عنصر معين في قائمة مرتبة تصاعدياً ، وتعتمد على فكرة تقسيم القائمة إلى نصفين بشكل متكرر حتى يتم العثور على العنصر المطلوب أو التأكد من عدم وجوده.

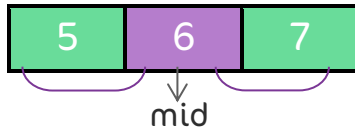
مثال توضيحي :



لنفرض لدينا المصفوفة التالية :

ولنفرض أننا نبحث عن العدد (7)

1. نقوم بتحديد الـ mid وهو (4) ونقارنه بالـ 7 فنجد أن $4 < 7$ لذا نتجاهل النصف الأيسر من المصفوفة كاملاً



7

2. نكرر العملية $6 < 7$ 3. نكرر العملية $7 = 7$ إذاً وجدنا العنصر المطلوب .

نلاحظ أنه :

في كل مرة تجاهلنا نصف المصفوفة أي نكون قد طبقنا Decrease by factor .

Decrease by random factor

مثال 5 :

لدينا مصفوفة A غير مرتبة ، لو رتبنا هذه المصفوفة تصاعدياً أوجد العنصر صاحب الترتيب k (ليكن $k=6$) ولكن ((الشرط هو عدم ترتيب المصفوفة)).



1. نختار عنصر بشكل عشوائي وليكن 8 مثلاً .

2. سنضع كل العناصر الأصغر منه قبله وكل العناصر الأكبر منه بعده .



3. هكذا أصبح العنصر 8 بمكانه الصحيح .

هل (8) أصبحت بالمكان السادس ($k=6$) ؟

كلا ، (8) في المكان الثامن

لذلك سنبحث عن العنصر صاحب المكان السادس ضمن العناصر الأصغر من 8



نختار رقم عشوائي وليكن 3 ونكرر الخطوات السابقة حتى يصبح 3 بترتيبه الصحيح .

1	2	3	5	7	4	6
---	---	---	---	---	---	---

العنصر (3) في المكان الثالث ، لذلك سنبحث عن العنصر صاحب المكان السادس و سيكون ضمن العناصر الأكبر من 3

A''	5	7	4	6
-----	---	---	---	---

العنصر صاحب المكان السادس هو سادس أصغر عنصر في A و A' ، وثالث أصغر عنصر في A''

نكرر الخطوات السابقة حتى نجد أن (6) هو العنصر صاحب الترتيب السادس (k=6) .

التعقيد في أفضل حالة هو : $O(n)$

التعقيد في أسوأ حالة هو : $O(n^2)$

■ ملاحظة :

بهذه الطريقة إما قد نجد الحل فوراً (لو اخترنا 6 من البداية) أو نصغر المسألة .
لكن مرة ستصغر المسألة عنصر واحد (لو اخترنا أكبر عنصر مثلاً) ومرة ستصغر للنصف أو قد نجد
العنصر مباشرةً وهكذا

لكن بكل الأحوال دائماً ستصغر المسألة لكن بقدر عشوائي ولهذا نقول :
Decrease by variable أو Decrease by random factor .

هذا النوع نادراً ما يتم استخدامه .

■ مثال 6 :

البحث عن كلمة في القاموس مثلاً set , cat .

- إذا كنا نبحث عن كلمة cat سنبدأ البحث بجزء من بداية القاموس
- أما كلمة set سننتقل للبحث فوراً بجزء قريب من نهاية القاموس
- أي : في كل مرة نقلل حجم المسألة بنسب مختلفة/عشوائية .

✍ ما الفائدة من وجود عدة طرق تفكير ؟

نختار الخوارزمية (طريقة التفكير) الأفضل حسب الـ trade-off .

■ ملاحظة : انتبه!!

- طريقة كتابة الكود مختلفة عن طريقة التفكير مثلاً في خوارزمية insertion sort :
- طريقة التفكير : Decrease and conquer .
- طريقة كتابة الكود : تكراري .

Turn your face toward the sun ..

Let the shadows fall behind you

The End